Chapter 9. Virtual memory. Full solutions

These original derivations correspond to the stable identifiers in the separate exercises PDF.

OS09-E01 Reference-string replacement

1. FIFO=9, LRU=10, OPT=7, CLOCK=9 faults. Initial free-frame loads fault; subsequent victims and hits follow each policy's own metadata.
2. FIFO victims are −,−,−,1,2,3,4,−,−,1,2,−. Accesses 8(page1), 9(page2), and 12(page5) hit without changing load order. The first two preserve [1,2,5]; the last preserves [5,3,4].
3. The same FIFO string faults 9 times with three frames and 10 with four. This is Belady anomaly, since FIFO lacks stack inclusion.
4. OPT still minimizes unit-cost misses for each fixed input and capacity using future information. FIFO's anomaly is not a counterexample to that separate optimality claim. Exact LRU updates recency on every access and needs counter/stack consistency, which is costly in hardware and concurrent metadata. Reference-bit methods approximate it without inheriting every theorem.

FIFO is load order, LRU oldest-to-newest, and CLOCK physical slots. Every state follows.

t/page	FIFO	LRU	OPT	CLOCK
1/1	[1]F	[1]F	[1]F	[1]F
2/2	[1,2]F	[1,2]F	[1,2]F	[1,2]F
3/3	[1,2,3]F	[1,2,3]F	[1,2,3]F	[1,2,3]F
4/4	[2,3,4]F	[2,3,4]F	[1,2,4]F	[4,2,3]F
5/1	[3,4,1]F	[3,4,1]F	[1,2,4]H	[4,1,3]F
6/2	[4,1,2]F	[4,1,2]F	[1,2,4]H	[4,1,2]F
7/5	[1,2,5]F	[1,2,5]F	[1,2,5]F	[5,1,2]F
8/1	[1,2,5]H	[2,5,1]H	[1,2,5]H	[5,1,2]H
9/2	[1,2,5]H	[5,1,2]H	[1,2,5]H	[5,1,2]H
10/3	[2,5,3]F	[1,2,3]F	[2,5,3]F	[5,3,2]F
11/4	[5,3,4]F	[2,3,4]F	[5,3,4]F	[5,3,4]F
12/5	[5,3,4]H	[3,4,5]F	[5,3,4]H	[5,3,4]H

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 9. Virtual memory. Full solutions (continued)

OS09-E02. Effective access time

1. Convert 5 ms to 5,000,000 ns. Then $EAT = (1 - p) \cdot 80 + p \cdot 5{,}000{,}000 = 80 + 4{,}999{,}920p$ ns. Thus $80 + 4{,}999{,}920p \leq 100$.
2. $p \leq 20/4{,}999{,}920$, approximately 4.00006×10^{-6}. This is approximately one fault per 250,000 references as a long-run fraction, not a deterministic spacing guarantee.
3. With 10,000,000 ns total fault cost, $p \leq 20/9{,}999{,}920$, approximately 2.000016×10^{-6}, or roughly one per 500,000 references as an average fraction.

Validation criterion. Convert units first and state whether the fault term includes completion of the reference.

OS09-E03. Working sets and thrashing

1. The sum is 72, exceeding 64 by 8 frames under a no-sharing count. Account for physically shared pages before interpreting the sum as exact frame demand.
2. No. Window length, low-frequency references, shared pages, and changing locality matter. Confirm sustained paging time and impaired useful progress using faults, service time, and execution evidence.
3. Adjust admission or active multiprogramming, allocations, reclaim/replacement policy, physical memory, or the workload's locality. Evaluate the effect rather than assuming one counter diagnoses the cause.

Validation criterion. A working-set estimate predicts pressure; runtime evidence establishes its consequences.

Chapter 9. Virtual memory. Full solutions (continued)

OS09-E04 Demand and COW faults

1. Verify VMA/read rights; revoke P1 PTE and complete old translation invalidation, including required remote shootdown; wait dirty writeback; read P5; publish the new mapping with required maintenance; retry. One fault, one 4096 B write, one 4096 B read,16 ms nonoverlapped I/O. Old-translation revocation is one maintenance phase, not an architecture-independent count of TLB instructions.
2. Clean P2 needs no writeback:one fault, zero disk writes, one disk read,8 ms without other overhead. Saving a write is not zero-copy.
3. Translation exists but COW temporarily forbids writes. Allocate F23, copy 4096 B while unpublished, install child writable F23, maintain TLB, retry 42. Parent remains read-only F17/41; F17 mappings 2→1. Child F23 has 42, refs 1, dirty. Parent may stay COW until its own later fault checks exclusive anonymous ownership.
4. Missing new PTE/translation installation causes refault; premature reuse while old mapping/TLB still reaches the frame corrupts another page; overwriting before dirty writeback completes loses old data. All are distinct from an authorized COW retry.

OS09-E05 Quotas and local/global victims

1. Raw 10.667/21.333/32; floors 10/21/32 total 63. Largest fraction A0.667 gets the last, final 11/21/32.
2. Local removes A2, giving A={1,7,3}, B={4,5}, C={6}. Global removes B4, giving A={1,2,3,7}, B={5}, C={6}. Both retain 6 frames, only global changes per-process counts.
3. B's next 4 faults after global eviction. A3 stays resident; deleting it to force A back to 3 would invent a second eviction. No event loads B8.
4. Fairness cannot create frames. Instruction/restart requirements may need code and operand pages simultaneously; satisfy the minimum, suspend/reject admission, or change execution. WSS is a performance requirement, not one universal correctness minimum. Kernel fault-handler pages require separate guarantees.

Chapter 9. Virtual memory. Full solutions (continued)

OS09-E06 Working sets and PFF

1. A={1,2,4,5}, B={6,7,8}, C={9}; WSS 4/3/1, D8>m6. If these distinct backing pages remain active, demand exceeds capacity. Shared frames instead require a unique-page union.
2. Rates 1.0>0.4 requests/grants a frame; 0.2 stays between thresholds; 0<0.1 permits spare reclaim under pressure and above minimum. Hysteresis prevents repeated add/remove at one threshold.
3. Reduce multiprogramming, suspend/swap, reallocate, or refuse new admission. CPU priority alone cannot solve absent residency.
4. Old references may no longer recur, pages may be shared, or owners may sleep. Low actual fault rate, high useful CPU, and stable paging queues do not support thrashing despiteD>m. High fault rate with low useful execution and growing paging I/O supports the diagnosis.

OS09-E07 Capacity monotonicity and offline optimality

1. Order seen pages newest→oldest, move each requested page to front. Induct on accesses:empty initially; hit changes order; miss inserts at front and evicts old item k. Thus k-frame LRU always holds the first k and Rk⊆R(k+1). A miss with k+1 implies absence from its subset Rk, so the smaller faults too. Sum pointwise indicators to obtainF(k+1)≤Fk for every string.
2. FIFO order follows each capacity's actual load/fault history, which changes with capacity. CLOCK's bits/hand/scans do too. Neither gives the same capacity-independent recency stack.
3. At the first shared-state divergence, an optimum evicts p while OPT evicts q with next(q)≥next(p). Modify the optimum to keep p/drop q. Before next(p), q is not requested earlier and common-page choices can be matched. If the original drops q sooner, align immediately; otherwise its next p faults while the modified schedule hits. Spend that saved transfer to restore q and align with its new state without more transfers. Early page-ins can be delayed to demand without cost increase. Iterating first divergences produces an optimum following OPT. Unit transfer cost is essential:extra dirty writes or heterogeneous page costs break the one-for-one exchange bound.

Chapter 9. Virtual memory. Full solutions (continued)

OS09-E08 COW and stale-TLB isolation

1. CPU1 can still store into F1 and corrupt P5. Revoke P1 PTE, complete local/remote invalidation acknowledgments, stop old accesses, and finish dirty I/O/remaining references before reuse. Only then load/install P5 and retry.
2. Initially both F40/7. P2 writes 9→new F53/9, while P1/cache/file stay 7; copy 4096 B. P1 later writes 8→new F52/8, while P2 remains F53/9 and file/cache 7; cumulative 8192 B. Private changes do not write the original file.
3. No immediate parent upgrade is required. Its later fault may upgrade without copy only after checking exclusively owned private anonymous data and no conflicting file-cache sharing, pins, or other references. One user mapping alone is insufficient.

OS09-E09 Conditional Linux 3.2 reclaim

1. P1 activate, P2 reclaim after removable mapping/refcounts, P3 start async writeback, P4 unevictable, P5 defer. Only P2 gives free+1; submitted I/O is not a free frame.
2. This dirty file branch requires kswapd and priority<DEF_PRIORITY−2. DEF_PRIORITY12 means 10 is blocked; direct reclaim is not kswapd. Retain/mark for other writeback rather than claiming this pageout started.
3. Set PageReferenced and returnPAGEREF_KEEP, keeping it inactive. First file reference is not unconditional activation; anon or prior PageReferenced changes the branch.
4. No. Mlock creates unevictable status; an extra DMA reference can instead fail final removal while remaining ordinary LRU. Writeback completion requires rechecking dirty/writeback, new references, pins, and mapping removal. New dirtiness or a remaining owner still prevents reuse.

Chapter 9. Virtual memory. Full solutions (continued)

OS09-E10 Solution: window versus capacity

(1) At t=4..8 the windows are [1,2,1,3], [2,1,3,1], [1,3,1,2], [3,1,2,4], [1,2,4,1]. Their sets are {1,2,3}, {1,2,3}, {1,2,3}, {1,2,3,4}, {1,2,4}; sizes are 3,3,3,4,3. At t=5 one page-1 reference expires and another enters, leaving count 2. Page 1 stays.
(2) Pre-access ranks are first, first, 2, first, 2, 3, first, 3. Capacity 2: F,F,H,F,H,F,F,F, totaling 6. Capacity 3: F,F,H,F,H,H,F,H, totaling 4. References 6 and 8 have rank 3, so only capacity 3 hits.
(3) After t=7, WS={1,2,3,4}, while LRU 3 holds [4,2,1]; page 3 has been evicted. Each distinct intervening page appears ahead of p, whereas repeated references to the same other page add no further page ahead. Hence r=d+1. The first k entries contain p iff $r \le k$, equivalently $d<k$. Handle first references as faults separately. Increasing k cannot turn a hit into a miss in this model; increasing a working-set window changes the demand estimate without directly allocating frames.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 9. Virtual memory. Full solutions (continued)

Verification code appendix: page_replacement.py

```
"""FIFO, LRU, and offline OPT page-replacement simulation.

Run:
    python page_replacement.py
"""


def run_policy(
    reference: list[int], frames: int, policy: str
) -> tuple[int, list[tuple[int, tuple[int, ...], bool, int | None]]]:
    resident: list[int] = []
    faults = 0
    trace: list[tuple[int, tuple[int, ...], bool, int | None]] = []
    for index, page in enumerate(reference):
        victim = None
        if page in resident:
            if policy == "LRU":
                resident.remove(page)
                resident.append(page)
            trace.append((page, tuple(resident), False, None))
            continue
        faults += 1
        if len(resident) < frames:
            resident.append(page)
            trace.append((page, tuple(resident), True, None))
            continue
        if policy == "FIFO":
            victim = resident.pop(0)
        elif policy == "LRU":
            victim = resident.pop(0)
        elif policy == "OPT":
            future = reference[index + 1 :]
            victim = max(
                resident,
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 9. Virtual memory. Full solutions (continued)

```
                key=lambda item: future.index(item) if item in future else len(future) + 1,
            )
            resident.remove(victim)
        else:
            raise ValueError(f"unknown policy: {policy}")
        resident.append(page)
        trace.append((page, tuple(resident), True, victim))
    return faults, trace


def simulate(reference: list[int], frames: int, policy: str) -> int:
    faults, _ = run_policy(reference, frames, policy)
    return faults


def main() -> None:
    reference = [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]
    result = {policy: simulate(reference, 3, policy) for policy in ("FIFO", "LRU", "OPT")}
    assert result["FIFO"] == 9
    assert result["LRU"] == 10
    assert result["OPT"] == 7
    lru_faults, lru_trace = run_policy(reference, 3, "LRU")
    victims = [entry[3] for entry in lru_trace if entry[3] is not None]
    assert lru_faults == 10
    assert victims == [1, 2, 3, 4, 5, 1, 2]
    assert [entry[3] for entry in lru_trace[9:]] == [5, 1, 2]
    print("faults", result)
    print("lru_trace", lru_trace)


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study